LANDIQ

Land-Acquisition Intelligence Platform

Architecture & Technology Overview

System architecture · Tech stack · Agents & orchestration
Retrieval-augmented answers · Evidence fusion · Failsafes
Data model · API · Deployment

Repository: RAGuard / landiq · Branch: version-3 · 24 October 2026

Status: prototype (single-instance, no authentication)

1. Executive overview

LANDIQ analyses bottlenecks in land acquisition for infrastructure corridors. It takes two kinds of input: **PDF documents** (case dossiers, awards, tribunal orders, notices, including scanned pages) and **GIS data** (GeoJSON, zipped Shapefile, GeoPackage, KML; map images as visual references). Two independent agents extract evidence. A persisted orchestrator coordinates them, the evidence is fused into acquisition cases joined only on verified identifiers, and the web interface shows each case with its risk, conflicts, recommended actions and a cited answer engine.

Design principles

- **Evidence first.** Every value shown for a real case links to a document page or map feature. Missing values stay *UNKNOWN*; disagreeing sources stay *CONFLICT*. Nothing is filled in by guesswork.
- **Deterministic control.** The orchestrator picks from a fixed whitelist of actions under step and time budgets. No language-model output selects actions, calls tools or changes job state.
- **Independent, fault-tolerant agents.** The PDF and GIS agents run concurrently and fail independently. Each input is retried, quarantined or degraded on its own, and checkpoints let a job resume after a restart.
- **Honest outputs.** Risk scores come from a trained model only if one is configured and evaluated; otherwise from a clearly labelled rule-based scorecard. No accuracy figures are claimed.
- **Replaceable infrastructure.** Storage sits behind a repository interface (SQLite today, PostgreSQL/PostGIS later); LLM providers sit behind a provider chain.

What a user does

Step	Where	What happens
1. Create project	Data & Analysis Jobs	A project groups the inputs and cases for one corridor.
2. Upload	Data & Analysis Jobs	PDFs and GIS files are validated at upload (type, size, content) and stored once (SHA-256 de-duplication).
3. Run analysis	Data & Analysis Jobs	A job runs in the background (HTTP 202). The page polls job state, agent status and failsafe status.
4. Answer questions	Job panel	If a parcel matches several cases, the user chooses; nothing is merged until then.
5. Review cases	Overview, High-Risk, Case Details, Map	Cases with statuses, risk, conflicts, map geometry and evidence links.
6. Ask & act	Case Details, Recommended Actions	Cited answers from the documents; actions recorded for human review (never executed).

2. Technology stack

Versions are those installed in the development environment for branch *version-3*.

Frontend

Technology	Version	Role
React + React DOM	19.2	Single-page application, component UI
TypeScript	6.0	Typed components and API client
Vite (+ @vitejs/plugin-react)	8.3	Dev server with /api proxy, production bundling
Tailwind CSS (@tailwindcss/vite)	4.3	Utility-first styling
Leaflet	1.9	GIS Parcel Map: parcels, alignment, popups
Recharts	3.10	Dashboard charts (risk distribution, evidence coverage, trend)
lucide-react	1.52	Icons
clsx, tailwind-merge	2.1 / 3.7	Class-name composition
oxlint	1.81	Linting

Backend

Technology	Version	Role
Python	3.12	Runtime
FastAPI + Uvicorn	0.162 / 0.54	REST API (/api/v1), OpenAPI docs, ASGI server; serves the built frontend
Pydantic + pydantic-settings	2.14	Schemas for every record and agent output; configuration from env / .env
PyMuPDF	2.0	PDF opening, validation, page text extraction, page rendering for OCR
RapidOCR (ONNX Runtime)	3.10 / 1.31	Local OCR with English PP-OCR models; no network needed
GeoPandas, pyogrio, Shapely, pyproj	1.2 / 0.13 / 2.2 / 3.8	GIS reading, CRS handling, reprojection, geometry validation, spatial relations
SQLite (WAL) + FTS5	built in	System of record and lexical (exact-term) index
ChromaDB	1.5	Vector store with local all-MiniLM-L6-v2 ONNX embeddings
google-genai	2.33	Gemini API: phrasing cited answers; vision OCR fallback
httpx	0.28	Ollama (local LLM) client; HTTP utilities
pytest	8+	80 tests incl. failure injection and restart recovery

Delivery

Item	Choice
Packaging	Multi-stage Dockerfile: Node 22 builds the frontend; Python 3.12-slim runs the backend and serves the bundle
Process model	Single container, single instance; background job runner (thread pool) inside the API process
State	One persistent volume: backend/storage (SQLite, uploads, quarantine, vector index, logs)
Configuration	Environment variables / .env (GEMINI_API_KEY, MODEL_PATH, RISK_SCORER, OCR_ENGINES, FAULTS, limits)

3. System architecture

The system has five layers. The browser talks only to the FastAPI layer. Analysis runs in a background job runner inside the same process, which drives the two agents, fusion and prediction, and persists every step. External LLM services are optional: without them, answers fall back to returning the cited evidence.

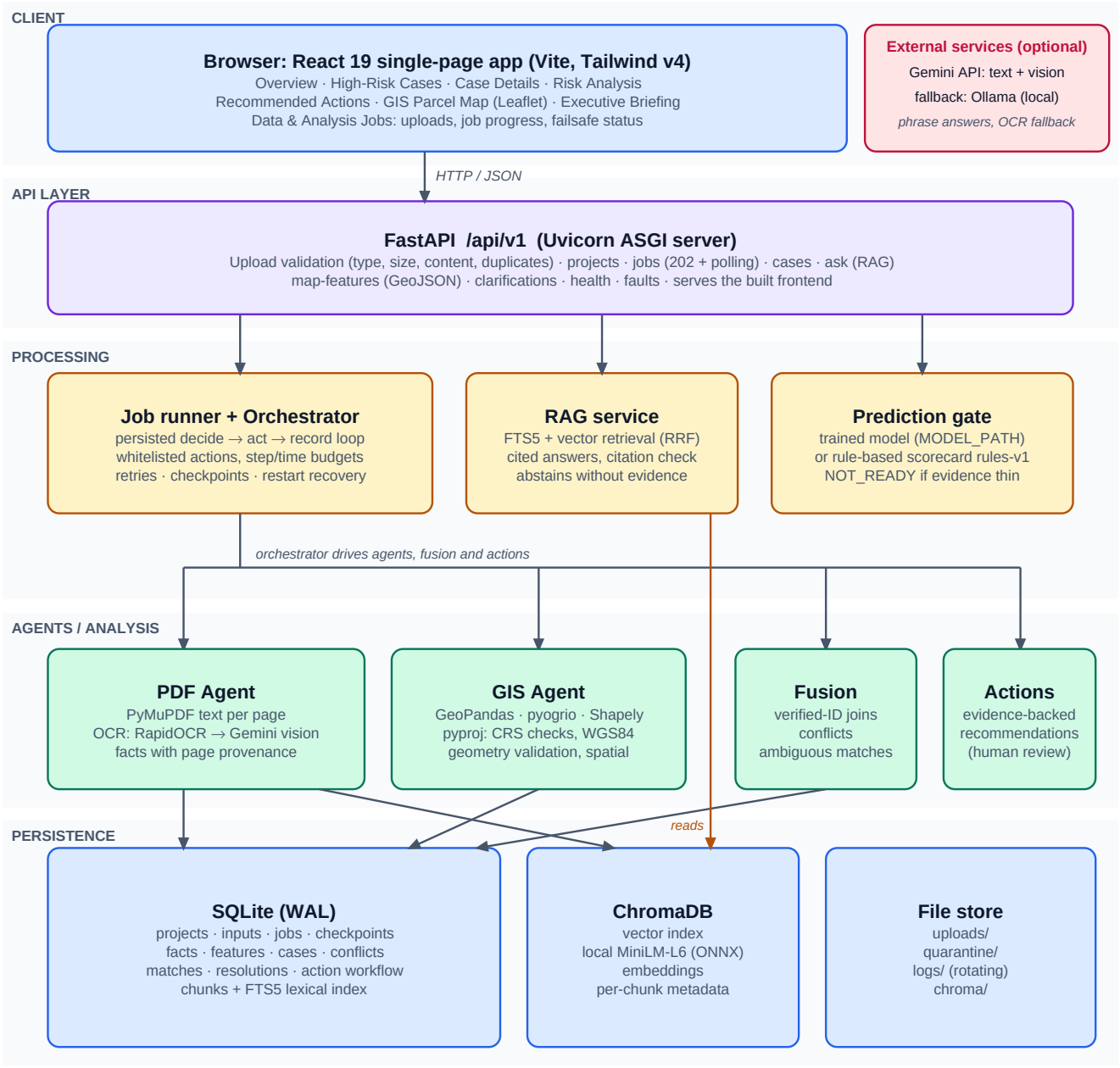

Figure 1. LANDIQ system architecture

Layer responsibilities

Layer	Components	Responsibility
Client	React SPA	Screens, polling of job state, uploads, map rendering, case review, clarification answers
API	api.py	Request validation, upload checks (HTTP 400/404/413/415), idempotent job creation, read models
Processing	orchestrator.py, rag.py, prediction.py	Job state machine, retries and checkpoints; grounded Q&A; risk scoring gate

Layer	Components	Responsibility
Agents / analysis	pdf_agent.py, ocr.py, map_agent.py, fusion.py, actions.py	Evidence extraction, spatial analysis, case building, recommendations
Persistence	storage.py, retrieval.py	SQLite repository and FTS5; ChromaDB vectors; file storage and quarantine

4. Backend modules

Module	Purpose
api.py	FastAPI app and routes; upload validation; startup (logging, demo seed, re-scoring stored cases, resuming interrupted jobs)
orchestrator.py	Persisted decide → act → record loop; per-input retries with backoff and timeouts; checkpoints; quarantine; failsafe events
pdf_agent.py	PyMuPDF extraction, case sectioning, rule-based fact extraction with page provenance, chunking
ocr.py	OCR chain for pages without text: RapidOCR (local) → vision LLM; line assembly; confidence filtering
map_agent.py	GIS reading, CRS validation and reprojection, geometry validation, feature roles, spatial relations, map facts
retrieval.py	Lexical (FTS5) + vector (ChromaDB) retrieval fused by reciprocal rank (k = 60); fallback rules
rag.py	Cited answers: evidence passed as numbered data, citations verified, abstention without evidence
llm.py	Provider chain (Gemini → Ollama) with retries, timeouts and cool-down; text and vision
fusion.py	Builds cases from verified identifiers; records conflicts and ambiguous parcel matches
prediction.py	Readiness gate; trained logistic bundle or rule-based scorecard; additive explanations
actions.py	Evidence-backed recommended actions with suggested owners, for human review
storage.py / runtime.py	SQLite repository (WAL) behind an interface; process-wide singletons
schemas.py	Pydantic models for projects, inputs, facts, features, cases, jobs, failsafe events, API payloads
faults.py	Fault injection used by tests and demos (agent crashes, outages, timeouts, invalid output)
demo.py	Synthetic demo corridor (840 cases) used for the read-only showcase project
config.py	Settings, limits, storage paths, logging

5. Analysis job pipeline

Creating a job returns HTTP 202 immediately; a worker thread then executes the job. At each step the orchestrator reads the persisted job, chooses the next whitelisted action, runs it and records the outcome in an audit trail. If the server restarts, RUNNING jobs resume and inputs that already have a checkpoint are not processed again.

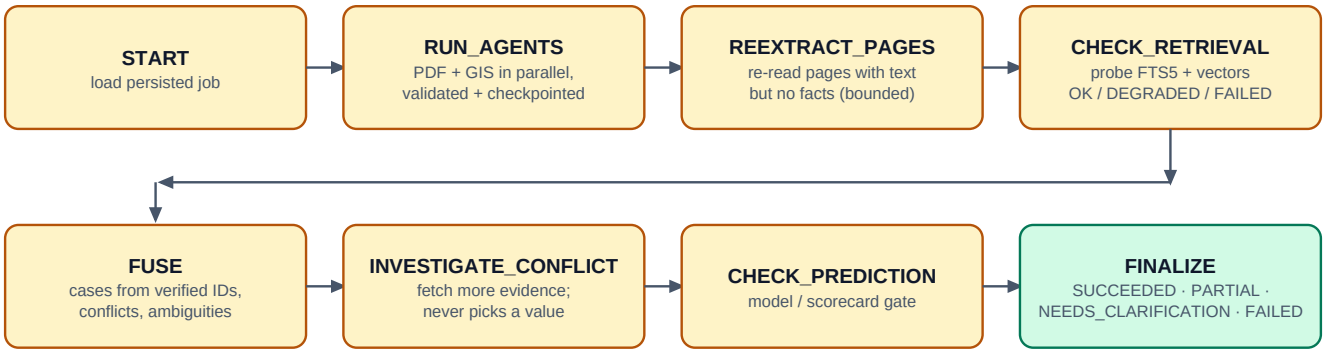

Figure 2. Orchestrator actions for one analysis job

Budgets bound every job: maximum steps (60), wall time (600 s), attempts per input (3), agent time per input (120 s), conflict investigations (10), page re-extractions (5), file sizes (50 MB PDF, 100 MB map), pages (500) and features (50,000).

6. PDF Agent (PyMuPDF)

The PDF agent turns each document into page-level facts and retrieval chunks. Facts from explicitly labelled lines (for example *Legal status: Dispute*) are **VALIDATED**; values inferred from free-text keywords are **UNVERIFIED**. A fact is attached to a case only when its section belongs to exactly one case reference. Document text is treated strictly as data: instructions written inside a PDF cannot change behaviour.

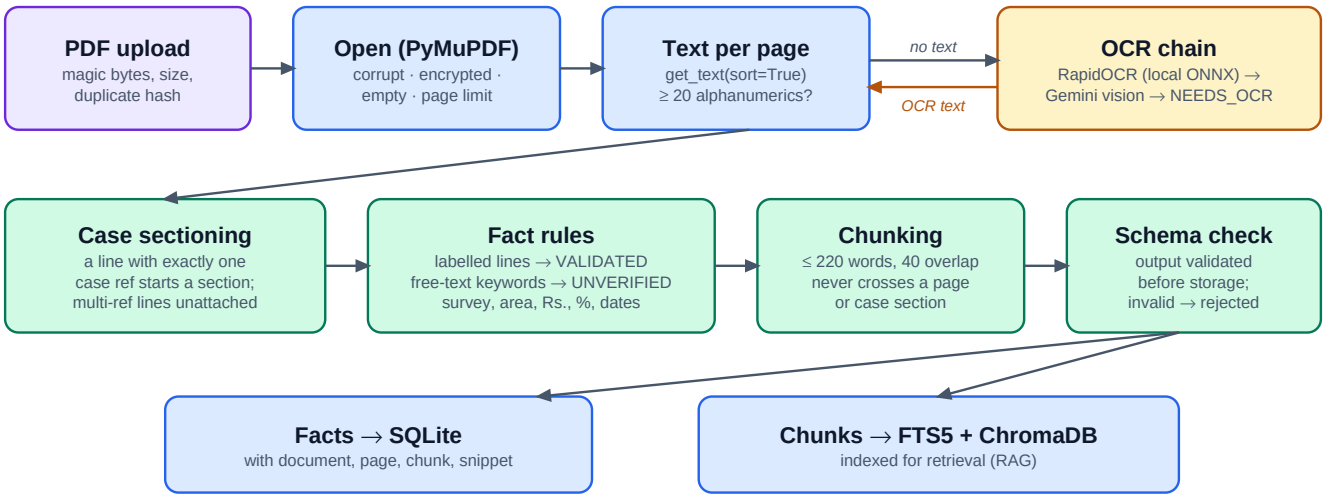

Figure 3. PDF agent processing flow

Extracted field	Examples
Identifiers	case refs (#0472), survey / khasra numbers, parcel IDs, village, district
Statuses	legal (CLEAR, DISPUTE, IN_COURT, PENDING_SETTLEMENT), compensation, approval/clearance, R&R
Quantities	area (ha), award amount (Rs. / lakh / crore → crore), disbursed %, R&R %, project progress %
Dates	notification, hearing, deadline

7. GIS Agent

The GIS agent never guesses a coordinate system. Layers without a CRS are rejected (GeoJSON and KML are WGS84 by specification), coordinates must fall inside valid longitude/latitude after reprojection, and invalid geometries are quarantined feature by feature. Line features named or attributed as an alignment/corridor are used to compute each parcel's distance to and intersection with the alignment, in a local UTM projection.

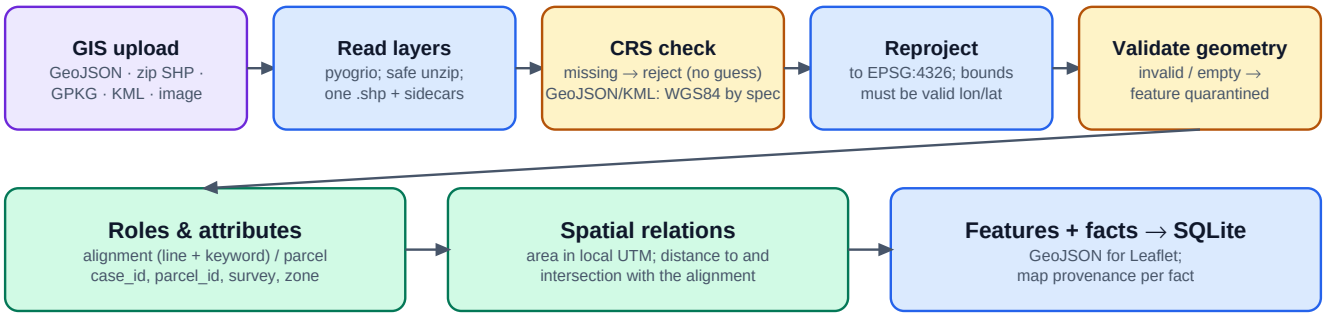

Figure 4. GIS agent processing flow

8. Retrieval-augmented answers (RAG)

- **Indexing.** Each chunk is stored in SQLite FTS5 (exact IDs, survey numbers, dates, amounts) and in ChromaDB (semantic similarity, local MiniLM embeddings), with project, document, page and case metadata.
- **Retrieval.** Both result lists are fused by reciprocal rank (k = 60) and filtered by project and case.
- **Answering.** Chunks are passed to the LLM as numbered, fenced evidence ([E1], [E2] …). Every claim must cite evidence; citations are checked against what was retrieved.
- **Abstention.** Uncited or wrongly cited answers are rejected and the evidence is returned instead (EVIDENCE_ONLY). With no usable evidence the answer is INSUFFICIENT_EVIDENCE.
- **Provider chain.** Gemini (primary and fallback model) → Ollama (local). A failing provider is skipped for a cool-down period; if all fail, the system returns evidence only.

9. Evidence fusion and risk scoring

Fusion joins facts and map features into cases only through verified identifiers: case references, exact parcel IDs and exact survey numbers. A parcel whose identifiers point at several cases is not merged; the job becomes NEEDS_CLARIFICATION and the user decides. Disagreeing validated values are kept as a CONFLICT with both sources. Recorded and mapped areas that differ by more than 10% raise a warning.

Risk scoring follows a readiness gate:

Source	When used	Output
Trained model bundle	MODEL_PATH points to an evaluated, fresh logistic model and every required feature is known	Delay probability, model version, exact additive explanation
Rule-based scorecard (rules-v1)	Default when no model is configured; needs at least 2 known status fields	Delay risk from fixed log-odds weights; labelled as not trained, no accuracy claimed
NOT_READY	Too little evidence, or RISK_SCORER=off, or a configured model is invalid/stale	No numbers

Scorecard field	Weights (log-odds)
Legal status	IN_COURT +2.0 · DISPUTE +1.6 · PENDING_SETTLEMENT +0.8 · CLEAR –0.6
Compensation status	DISPUTED +1.3 · PENDING +1.0 · IN_PROGRESS +0.4 · DISBURSED –0.6
Approval / clearance	DELAYED +1.2 · PENDING +0.8 · UNDER_REVIEW +0.5 · CLEARED –0.5
R&R status	PENDING +0.6 · IN_PROGRESS +0.3 · COMPLETED –0.4
Adjustments	intercept –1.5 · conflicting field +0.8 · unverified value × 0.5 · bands: HIGH ≥ 65%, MEDIUM ≥ 35%

10. Fault tolerance and failsafes

Every agent error is classified, handled for that input only, and recorded as a **failsafe event** with the agent, input, trigger, reason, fallback used and recovery outcome (SUCCEEDED, PARTIAL or FAILED). The Data & Analysis Jobs page shows a status card per agent; with no event it reads *Failsafe Triggered: NO.*

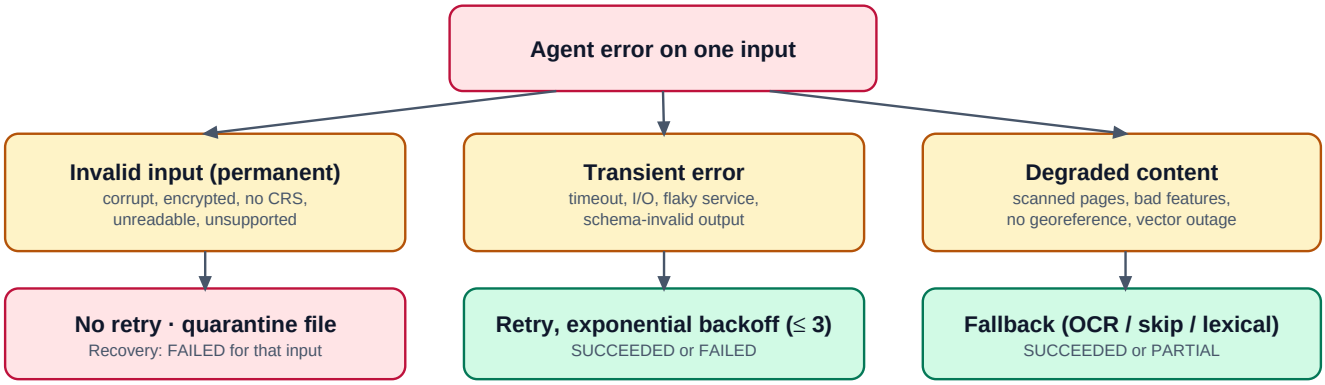

Figure 5. Failsafe classes and recovery outcomes

Agent	Trigger	Fallback	Recovery
PDF	TRANSIENT_ERROR / AGENT_TIMEOUT / AGENT_OUTPUT_INVALID	Retry with exponential backoff (max 3)	Succeeded / Failed
PDF	OCR_USED	Scanned pages read by RapidOCR, then Gemini vision	Succeeded
PDF	PAGES_UNREADABLE	Unreadable pages skipped, rest of the document used	Partial
PDF	NEEDS_OCR	All OCR engines tried; document skipped, nothing guessed	Failed
PDF	PDF_CORRUPT / PDF_ENCRYPTED / PDF_EMPTY	No retry; file quarantined	Failed
GIS	GEOMETRY_INVALID (some features)	Invalid features quarantined, valid ones kept	Partial
GIS	NO_GEOREFERENCE	Image kept as visual reference only	Partial
GIS	CRS_MISSING / CRS_INVALID / MAP_UNREADABLE / MAP_INVALID_ARCHIVE / MAP_EMPTY	No retry; file quarantined	Failed
Both	AGENT_CRASH	Crash isolated; other inputs and the other agent continue	Failed

System-level situation	Behaviour
Invalid upload (not a PDF, empty, unsupported, oversized)	Rejected before storage with HTTP 400 / 413 / 415
Vector store down	Lexical-only retrieval (DEGRADED); if lexical also fails, answers abstain
LLM down or uncited answer	Evidence-only answer; nothing unsupported is shown
Conflicting sources / ambiguous parcel	CONFLICT kept with both sources / NEEDS_CLARIFICATION until the user answers
Server restart	Running jobs resume; checkpointed inputs are not re-processed
Instructions inside documents	Treated as quoted data; never change behaviour

11. Data model

Entity	Key fields	Notes
Project	project_id, name, corridor_id, is_demo	Groups inputs, jobs and cases
Input	input_id, kind (document/map), filename, sha256, status	STORED, DUPLICATE or QUARANTINED
Job	state, agent statuses, retrieval/fusion stages, audit, clarifications	Persisted after every step; resumable
AgentStatus	state, attempts, errors, warnings, failsafes	One per agent per job
FailsafeEvent	agent, trigger, input, reason, fallback, recovery	Shown in the Failsafe status cards
Checkpoint	job_id, agent, input_id, result	Validated agent output per input
Fact	field, value, normalized_value, case_id, validation_status, provenance	Provenance: document page/chunk/snippet or map feature
MapFeature	role, geometry (WGS84), parcel_id, case_id, survey numbers, area, alignment relation	Leaflet-ready GeoJSON
Case	case_ref, fields (value + status + facts), spatial, conflicts, prediction, actions	Field status: KNOWN, UNVERIFIED, UNKNOWN, CONFLICT
Conflict / UnresolvedMatch	field + competing values / candidate cases + question	Never resolved automatically
Chunk	chunk_id, document, page, text, case_ids	In FTS5 and ChromaDB

12. REST API (/api/v1)

Method	Route	Purpose
GET	/health	Retrieval health, prediction readiness, active faults
GET / POST	/projects	List / create projects
GET	/projects/{id}/summary	Counts by risk band and evidence quality
POST	/projects/{id}/documents · /maps	Upload PDF / GIS file (201, or 400/404/413/415)
GET	/projects/{id}/inputs	Uploaded files and status
POST	/projects/{id}/analysis-jobs	Start a job (202, idempotent via Idempotency-Key)
GET	/projects/{id}/jobs · /jobs/{job_id} · /jobs/{job_id}/results	Job state, agents, failsafes, audit; results
POST	/jobs/{job_id}/cancel	Cancel a running job
GET	/projects/{id}/cases · /cases/{case_id}	Cases; detail with facts, provenance, conflicts
POST	/cases/{case_id}/ask · /projects/{id}/ask	Cited RAG answer
GET	/projects/{id}/map-features	GeoJSON for the map (EPSG:4326)
POST	/projects/{id}/clarifications	Answer an ambiguous parcel match
POST	/actions/{action_id}/workflow	Record review status of an action
GET / PUT	/faults	List / set simulated faults (disable with ALLOW_FAULT_INJECTION=false)

13. Frontend architecture

The frontend is a single React application. *App.tsx* holds the selected project and page; *api/client.ts* is a typed client for every endpoint and *api/adapter.ts* converts backend records into view models (field labels, risk bands, evidence). Pages are plain components; there is no client-side router or global state library.

Page	Shows
Overview (Dashboard)	Case counts, risk bands, evidence coverage, cases needing attention
High-Risk Cases	Ranked case queue with filters (risk, zone, search)
Case Details	Statuses with evidence snippets, conflicts, risk panel, cited Q&A
Risk Analysis & SHAP	Additive contributions from the model or scorecard
Recommended Actions	Evidence-backed actions with review workflow
GIS Parcel Map	Leaflet map of parcels and alignment, coloured by risk or evidence quality
Executive Briefing	Corridor summary and downloadable text report with limitations
Data & Analysis Jobs	Project creation, uploads with rejection reasons, job progress, failsafe status per agent, audit trail, clarification questions

14. Deployment and operations

- **Local development:** Uvicorn on port 8000 and Vite on port 5173 (proxying /api). Single-server mode: build the frontend and the backend serves it.
- **Container:** *docker build -t landiq ./landiq*, run with a volume for *backend/storage* and GEMINI_API_KEY as an environment variable. Health check on /api/v1/health.
- **Logging:** rotating log files in storage/logs; errors returned to clients carry a reference ID, never stack traces.
- **Testing:** 80 backend tests (agents, OCR, scoring, orchestration, failure injection, restart recovery, API); frontend type-check, build and lint.

15. Current limitations and next steps

Limitation	Planned direction
Single instance (SQLite, in-process job runner)	PostgreSQL/PostGIS behind storage.py; external job queue for scale-out
No authentication or multi-tenancy	Authenticating reverse proxy / SSO; per-tenant data separation
No trained prediction model	Collect labelled historical outcomes; train, evaluate and configure via MODEL_PATH
Rule-based fact extraction	Additional document layouts; reviewed LLM-assisted extraction with the same validation gates
Georeferenced rasters / GeoPDF not parsed	Raster georeferencing support
Frontend bundle ~900 KB	Route-level code splitting